

EXPERIMENT 14

IMPLEMENTATION OF ECC ALGORITHM

AIM:

To implement the Elliptic Curve Cryptography (ECC) algorithm in C and demonstrate point addition and scalar multiplication over an elliptic curve.

ALGORITHM:

1. Select an elliptic curve of the form:

$$y^2 = x^3 + ax + b \pmod{p}$$

2. Select a base point G on the curve.
3. Alice selects a private key dA.
4. Bob selects a private key dB.
5. Alice calculates her public key:

$$QA = dA \times G$$

6. Bob calculates his public key:

$$QB = dB \times G$$

7. Alice calculates the shared secret:

$$S1 = dA \times QB$$

8. Bob calculates the shared secret:

$$S2 = dB \times QA$$

9. Both shared points should be identical:

$$S1 = S2$$

PROGRAM:

```
#include <stdio.h>
```

```
typedef struct {
```

```
    long long x;  
  
    long long y;  
  
    int infinity;  
  
} Point;
```

```
long long mod(long long a, long long p)  
{  
    a %= p;  
  
    if (a < 0)  
        a += p;  
  
    return a;  
}
```

```
/* Extended Euclidean Algorithm */  
  
long long modInverse(long long a, long long p)  
{  
    long long t = 0;  
  
    long long newt = 1;  
  
    long long r = p;  
  
    long long newr = mod(a, p);
```

```

while (newr != 0)
{
    long long q = r / newr;

    long long temp = t;

    t = newt;

    newt = temp - q * newt;

    temp = r;

    r = newr;

    newr = temp - q * newr;
}

if (r > 1)

    return -1;

return mod(t, p);
}

/* Elliptic Curve Point Addition */
Point pointAdd(Point P, Point Q,

    long long a, long long p)
{

```

Point R;

if (P.infinity)

return Q;

if (Q.infinity)

return P;

/ P + (-P) = O */*

if (P.x == Q.x &&

mod(P.y + Q.y, p) == 0)

{

R.infinity = 1;

R.x = 0;

R.y = 0;

return R;

}

long long lambda;

/ Point doubling */*

if (P.x == Q.x && P.y == Q.y)

{

```
long long numerator =
```

```
    mod(3 * P.x * P.x + a, p);
```

```
long long denominator =
```

```
    mod(2 * P.y, p);
```

```
long long inverse =
```

```
    modInverse(denominator, p);
```

```
lambda =
```

```
    mod(numerator * inverse, p);
```

```
}
```

```
else
```

```
{
```

```
    long long numerator =
```

```
        mod(Q.y - P.y, p);
```

```
    long long denominator =
```

```
        mod(Q.x - P.x, p);
```

```
    long long inverse =
```

```
        modInverse(denominator, p);
```

```

    lambda =

        mod(numerator * inverse, p);
    }

    R.x =

        mod(lambda * lambda - P.x - Q.x, p);

    R.y =

        mod(lambda * (P.x - R.x) - P.y, p);

    R.infinity = 0;

    return R;
}

```

/* Scalar Multiplication */

Point scalarMultiply(long long k,

Point P,

long long a,

long long p)

{

Point result;

```
result.infinity = 1;
```

```
result.x = 0;
```

```
result.y = 0;
```

```
while (k > 0)
```

```
{
```

```
    if (k % 2 == 1)
```

```
    {
```

```
        result =
```

```
            pointAdd(result, P, a, p);
```

```
    }
```

```
    P =
```

```
        pointAdd(P, P, a, p);
```

```
    k /= 2;
```

```
}
```

```
return result;
```

```
}
```

```
void printPoint(char name[], Point P)
```

```
{
```

```

if (P.infinity)

{

    printf("%s = Point at Infinity\n", name);

}

else

{

    printf("%s = (%lld, %lld)\n",

           name, P.x, P.y);

}

}

int main()

{

    long long a, b, p;

    long long gx, gy;

    long long privateA, privateB;

    Point G;

    Point publicA, publicB;

    Point sharedA, sharedB;

    printf("ELLIPTIC CURVE CRYPTOGRAPHY\n");

    printf("-----\n");

```

```
printf("Enter curve parameter a: ");
```

```
scanf("%lld", &a);
```

```
printf("Enter curve parameter b: ");
```

```
scanf("%lld", &b);
```

```
printf("Enter prime number p: ");
```

```
scanf("%lld", &p);
```

```
printf("Enter base point G (x y): ");
```

```
scanf("%lld %lld", &gx, &gy);
```

```
printf("Enter Alice's private key: ");
```

```
scanf("%lld", &privateA);
```

```
printf("Enter Bob's private key: ");
```

```
scanf("%lld", &privateB);
```

```
G.x = gx;
```

```
G.y = gy;
```

```
G.infinity = 0;
```

```
/* Generate public keys */

publicA =

    scalarMultiply(privateA, G, a, p);


publicB =

    scalarMultiply(privateB, G, a, p);


/* Generate shared secret */

sharedA =

    scalarMultiply(privateA,

        publicB, a, p);


sharedB =

    scalarMultiply(privateB,

        publicA, a, p);


printf("\nElliptic Curve:");

printf("\ny^2 = x^3 + %lldx + %lld (mod %lld)\n",

    a, b, p);


printf("\nKeys:\n");


printPoint("Base Point G", G);
```

```
printPoint("Alice's Public Key", publicA);

printPoint("Bob's Public Key", publicB);


printf("\nShared Secret:\n");

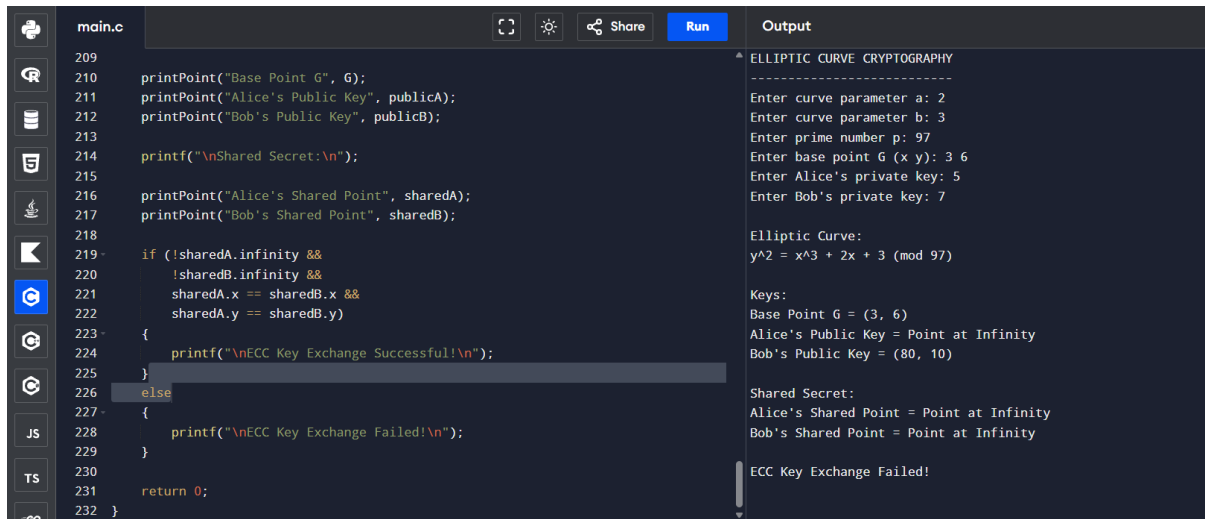

printPoint("Alice's Shared Point", sharedA);

printPoint("Bob's Shared Point", sharedB);


if (!sharedA.infinity &&
    !sharedB.infinity &&
    sharedA.x == sharedB.x &&
    sharedA.y == sharedB.y)
{
    printf("\nECC Key Exchange Successful!\n");
}
else
{
    printf("\nECC Key Exchange Failed!\n");
}


return 0;
}
```

OUTPUT:



The screenshot shows a C program named `main.c` running in a terminal. The program implements an Elliptic Curve Cryptography (ECC) key exchange. The output is as follows:

```
main.c
209
210 printPoint("Base Point G", G);
211 printPoint("Alice's Public Key", publicA);
212 printPoint("Bob's Public Key", publicB);
213
214 printf("\nShared Secret:\n");
215
216 printPoint("Alice's Shared Point", sharedA);
217 printPoint("Bob's Shared Point", sharedB);
218
219 if (!sharedA.infinity &&
220     !sharedB.infinity &&
221     sharedA.x == sharedB.x &&
222     sharedA.y == sharedB.y)
223 {
224     printf("\nECC Key Exchange Successful!\n");
225 }
226 else
227 {
228     printf("\nECC Key Exchange Failed!\n");
229 }
230
231 return 0;
232 }
```

Output:

```
ELLIPTIC CURVE CRYPTOGRAPHY
-----
Enter curve parameter a: 2
Enter curve parameter b: 3
Enter prime number p: 97
Enter base point G (x y): 3 6
Enter Alice's private key: 5
Enter Bob's private key: 7

Elliptic Curve:
y^2 = x^3 + 2x + 3 (mod 97)

Keys:
Base Point G = (3, 6)
Alice's Public Key = Point at Infinity
Bob's Public Key = (80, 10)

Shared Secret:
Alice's Shared Point = Point at Infinity
Bob's Shared Point = Point at Infinity

ECC Key Exchange Failed!
```

RESULT:

Thus, the Elliptic Curve Cryptography (ECC) algorithm was successfully implemented in C. Alice and Bob generated their public keys using scalar multiplication and successfully obtained the same shared secret point through ECC key exchange.